

Grupo 4 — PG6 U4 — Asistente LangChain (AFPI)

Autor/es: [Díaz, Ignacio; Pozzoli, Gonzalo; Manrique, Martín; Veggiani, Franco]

Resumen Asistente inteligente en español para consultas de finanzas personales (AFPI), basado en un LLM open-source desplegado con Ollama e integrado con Neo4j mediante LangChain. Traduce preguntas en lenguaje natural a Cypher seguro, ejecuta sobre el grafo, y redacta una respuesta breve consistente con los datos.

Índice

- Objetivo
 - Propósito funcional del asistente
 - Despliegue del modelo LLM con Ollama
 - Base de conocimiento en Neo4j
 - Integración con LangChain (flujo + fragmento de código)
 - Pruebas y evaluación
 - Diagrama/captura del grafo
 - Reflexión grupal y mejoras
 - Anexos (configuración y uso)
-

Objetivo

Desarrollar un asistente inteligente que utilice un modelo de lenguaje open source (Ollama) integrado a una base de conocimiento (Neo4j) mediante LangChain, capaz de responder en lenguaje natural a consultas sobre finanzas personales (AFPI: usuarios, movimientos, categorías, objetivos y recomendaciones).

Propósito funcional del asistente

- ¿A quién asiste?
 - Usuarios finales y analistas que desean entender su situación financiera: movimientos, gastos por categoría, recomendaciones y progreso de objetivos.
 - ¿Qué tipo de consultas responde?
 - Listado y filtrado de movimientos por fecha/tipo/categoría.
 - Resúmenes: ingresos vs gastos, balance, gasto por categoría en períodos.
 - Estado de objetivos de ahorro: meta, progreso, urgencia.
 - Recomendaciones activas: tipo, texto, categoría base, objetivo relacionado.
 - Ejemplos de consultas en lenguaje natural:
 - "Listá los últimos 10 movimientos de user#maria con fecha, tipo, monto y categoría."
 - "Top 3 categorías por gasto de user#maria este mes, con totales."
 - "¿Qué recomendaciones de tipo 'Alerta' tiene user#carlos?"
 - "Resumen financiero de user#maria (ingresos, gastos, balance) en octubre 2025."
-

Despliegue del modelo LLM con Ollama

- Instalación (Linux/macOS/Windows): descargar desde <https://ollama.com/download>
 - Descarga de modelo y verificación:
 - `ollama pull llama3.2`
 - `ollama run llama3.2` (probar que responde)
 - Opciones livianas: `llama3:3b-instruct`, `qwen:3b-instruct`, `gemma:2b`
 - Ejecución como servicio local:
 - `ollama serve`
 - Configuración en `.env`:
 - `OLLAMA_MODEL=llama3.2` (o el que se prefiera)
-

Base de conocimiento en Neo4j

- Modelado (metamodelo AFPI)
 - Nodos:
 - Frame {name}
 - Slot {name}
 - FrameInst {id}
 - Relaciones clave:
 - (FrameInst)-[:INSTANCE_OF]->(Frame)
 - (FrameInst)-[:HAS_VALUE {slot, value, ts?}]->(Slot)
 - (Usuario:FrameInst)-[:REALIZA]->(Movimiento:FrameInst)
 - (Movimiento)-[:PERTENECE_A]->(Categoría:FrameInst)
 - (Usuario)-[:TIENE]->(Objetivo:FrameInst)
 - (Usuario)-[:RECIBE]->(Recomendación:FrameInst)
 - (Recomendación)-[:BASADO_EN]->(Categoría), (Recomendación)-[:DIRIGIDA_A]->(Objetivo)
 - Esquema y datos
 - `SCHEME.txt`: constraints, catálogo básico y triggers APOC (frecuencia/nivel de gasto, umbral, urgencia/progreso).
 - `DATOS.txt`: usuarios (María/Carlos), categorías, movimientos (con fechas/montos), objetivos de ahorro y recomendaciones.
 - Carga en Neo4j Desktop
 - Abrir Browser de la DB.
 - Ejecutar `SCHEME.txt` primero (crea estructura y triggers).
 - Ejecutar `DATOS.txt` luego (puebla entidades y relaciones).
 - Consultas semánticas de prueba (ejemplos típicos)
 - Gasto total por categoría (30 días), ingresos vs gastos por día, objetivos con progreso, recomendaciones por tipo.
-

Integración con LangChain

- Flujo del asistente
 - Entrada: pregunta en español.

- Prompting: contexto del esquema + ejemplos (few-shots) + restricciones de seguridad.
 - Generación de Cypher (LLM via LangChain ChatOllama).
 - Validación de seguridad (solo lectura).
 - Ejecución Cypher en Neo4j y tabulado de resultados.
 - Redacción de respuesta breve basada exclusivamente en resultados.
 - Fragmento de código (resumen)
 - Cliente LLM y conexión Neo4j (nl2cypher.py):
 - `ChatOllama(model=..., temperature=0.0)`
 - `GraphDatabase.driver(NEO4J_URI, auth=...)`
 - Prompt con esquema + few-shots (afpi_context.py + nl2cypher.py):
 - Carga del esquema desde `SCHEME.txt` y “auto-introspección” a la DB (Frames/Slots/Relaciones).
 - Few-shots parseados automáticamente desde “CONSULTAS DE EJEMPLO...” en `DATOS.txt`.
 - Seguridad (solo lectura):
 - Rechaza keywords peligrosas (CREATE, MERGE, DELETE, CALL, APOC, etc.) y exige `RETURN`.
 - Respuesta en texto:
 - Convierte el resultado a CSV y redacta una respuesta breve fiel a los datos.
 - Nota sobre el “retriever”
 - El “retrieval” se realiza vía traducción NL → Cypher. El LLM genera una consulta que “recupera” la evidencia directamente del grafo (sin embeddings documentales). Es una forma de RAG estructural sobre Neo4j.
-

Pruebas y evaluación

- Set up de pruebas
 - Neo4j Desktop con `SCHEME.txt` + `DATOS.txt` cargados.
 - Ollama en ejecución con `OLLAMA_MODEL` disponible.
 - Scripts:
 - `smoke_test.py`: verificación rápida sin LLM (3 queries determinísticas).
 - `nl2cypher.py`: interacción completa NL → Cypher.
- Escenarios de interacción (mínimo 3)
 - Escenario A: Resumen financiero del mes
 - Prompt: “Resumen financiero de user#maria este mes (ingresos, gastos y balance).”
 - Expectativa: tabla con totales y balance coherente con sus movimientos.
 - Criterio: exactitud de agregaciones, sin mezclar con otro usuario.
 - Escenario B: Top categorías por gasto
 - Prompt: “Top 3 categorías por gasto de user#carlos en los últimos 30 días.”
 - Expectativa: categorías con sumas descendentes; descarta Ingresos.
 - Criterio: filtro de tipoDeMovimiento = ‘Gasto’, rango temporal aplicado.

- Escenario C: Recomendaciones activas de tipo “Alerta”
 - Prompt: “Mostrame las recomendaciones de tipo ‘Alerta’ para user#maria.”
 - Expectativa: lista de recomendaciones con su descripción y categoría base.
 - Criterio: coincidencia del tipo, columnas correctas y texto íntegro.
- Métricas cualitativas
 - Precisión semántica (interpretación correcta de la intención).
 - Factualidad (coherencia con datos del grafo).
 - Estabilidad del Cypher (uso de patrones AFPI, sin verbos prohibidos).
 - Utilidad de la respuesta (brevedad, columnas relevantes, ordenamiento).
- Resultados (resumen)
 - El LLM generó consultas correctas en la mayoría de los casos con pocos-shots AFPI.
 - El validador evitó operaciones de escritura/DDL.
 - Las respuestas en español fueron breves y fieles al resultado de la tabla.
- Oportunidades de mejora
 - Aumentar cobertura de few-shots (fechas relativas, combinaciones de filtros).
 - Afinar el “post-prompt” para responder con vocabulario más uniforme.
 - Implementar cacheado de consultas/respuestas (evitar latencia).

Diagrama o captura del grafo

Insertar una de las siguientes (o ambas):

- Captura del Browser Neo4j mostrando nodos **Frame**, **Slot**, **FrameInst** y relaciones.
- Diagrama conceptual del metamodelo AFPI:
 - FrameInst —INSTANCE_OF→ Frame
 - FrameInst —HAS_VALUE→ Slot (value/slot/ts)
 - Usuario —REALIZA→ Movimiento —PERTENECE_A→ Categoría
 - Usuario —TIENE→ Objetivo
 - Usuario —RECIBE→ Recomendación —BASADO_EN→ Categoría —DIRIGIDA_A→ Objetivo

Reflexión grupal sobre resultados y mejoras

- Qué funcionó bien
 - La combinación de pocos-shots específicos + contexto del esquema elevó la precisión del Cypher.
 - La seguridad por lista de verbos prohibidos evitó modificaciones accidentales del grafo.
 - El enfoque NL→ Cypher resultó suficiente para responder preguntas frecuentes sin un índice vectorial.
- Desafíos

- Fechas relativas (“este mes”, “últimos 30 días”) requieren ejemplos consistentes para que el LLM genere filtros correctos.
 - Algunos prompts muy genéricos derivan en Cypher ambiguo; conviene educar al usuario con ejemplos.
 - Mejoras futuras
 - Incluir “function calling” o plantillas de Cypher con variables para controlar mejor el SQL/Cypher generado.
 - Añadir un paso de verificación automática (parsing/linting de Cypher).
 - Integrar un pequeño catálogo de sinónimos (e.g., “gasto de supermercado” ~ “categoría Supermercado”).
 - Evaluaciones automáticas (benchmarks de prompts y expected queries).
-

Anexos

- Configuración `.env` (ejemplo)
 - `NEO4J_URI=neo4j://localhost:7687`
 - `NEO4J_USER=neo4j`
 - `NEO4J_PASS=*****`
 - `NEO4J_AUTH_DISABLED=` (vacío si hay auth; `true` si no)
 - `OLLAMA_MODEL=llama3.2`
 - Comandos útiles
 - Validación sin LLM: `.venv/bin/python smoke_test.py`
 - Asistente NL → Cypher: `.venv/bin/python nl2cypher.py`
 - Fragmento de código (referencias)
 - `nl2cypher.py`: generación/validación/ejecución y respuesta en NL.
 - `afpi_context.py`: construcción de contexto de esquema (SCHEME + introspección) y few-shots desde DATOS.
-

Fin del documento.